

Evaluating Feed Forward Neural Networks on Classification and Regression Tasks

FYS-STK4155 - Project 2

Erik Berthelsen, Morten Taraldsten Brunes, Johanna Annie Kristina Tjernström
Department of Geoscience, University of Oslo

Feed Forward Neural Networks (FFNN) are a fundamental tools in modern day machine learning, significantly advancing the capabilities of data analysis. Such networks are capable of capturing complex, non-linear relationships in data that linear models cannot. There is a need to broaden the understanding and knowledge of neural networks, the use of different hyperparameters and it's performance on regression and classification tasks. We implemented our own feed forward neural network code with flexibility of varying input features, predictors, different activation and cost functions, as well as penalty terms. Regression was analyzed using the Runge function, while classification was performed on the widely used MNIST database of handwritten digits. Both tasks were executed with our own code and PyTorch for a comparative analysis. This work builds upon prior knowledge of OLS, Ridge and Lasso regression and gradient descent methods acquired in a previous study. Results show that choice of hyperparameters are important and must be tailored to the dataset. Consequently, several numerical analysis with different hyperparameters must be conducted to find the model that generalized the best to unseen data, regardless of whether the task is regression or classification. PyTorch performs slightly better than our FFNN, documenting some differences in implementation.

I. INTRODUCTION

Machine learning models provide the ability to learn and model non-linear patterns in complex datasets, where traditional approaches often fail. Nowadays large amount of data are available and machine learning has developed to algorithms that derive information from data. A subset of machine learning is supervised learning which use labeled data to train a predictive model, which is used to make predictions on future data. This method can be used for discrete classification and continuous regression tasks [7]. Feed forward neural networks uses a defined mapping $y = f(x; \theta)$ to capture these relationships by learning the best values for the parameters θ that results in the best approximation of the true function [3]. However, their dependence on hyperparameters and architecture strongly affects the performance of such models.

In this report we consider regression on the Runge phenomenon [1] and classification on the well known MNIST database of handwritten digits [6]. For regression we used the same dataset as i project 1 to do comparative studies of analytical and neural network. We have coded our own implementation of FFNN with different optimizers, cost functions and norms, while also being flexible to the number of hidden layers, input features and predictors. Comparative studies were done between our own code and open source python libraries such as PyTorch, to ensure that numeric computation were correct. Analysis were performed to evaluate the optimal hyperparameters to training the models.

The methods used in this report are described in detail in section II. The results of regression and classification with FFNN, as well as any insights are presented in III A and III B, followed by a general discussion of the FFNN. Finally, a concluding part will be presented in IV. Code availability will be available in V.

II. METHODS

A. Datasets and preprocessing

1. Runge function

For this project we generated a synthetic dataset based on the one-dimensional Runge function, to ensure that the same settings and preprocessing we used for both the neural network and testing:

$$f(x) = \frac{1}{1 + 25x^2} \quad (1)$$

Where the design matrix, X , is created by $[x_1, x_2, x_3 \dots]$ on $x \in [-1, 1]$, drawn on $n = 1000$ data points and from two targets; one without noise, we set the standard deviation to zero to get the true function, and one noisy target where we added a standard deviation of 0.1. The noise-free target is kept for comparison, while the noisy target was used for the training.

The Runge function is a problem of oscillation at the edges of an interval that occurs when using polynomial interpolation of higher degree over a set of evenly spaced interpolation points. The Runge function shows that higher polynomial degrees does not always improve accuracy [1]. We use our own implementation of a feed forward neural network, in this paper noted as FFNN, and validate that performance with PyTorch for both regression and classification task. The goal is to validate the performance of FFNN, and analyze how different hyperparameters influence the results of FFNN. We will also compare FFNN results to OLS, Ridge and Lasso results from project 1[9].

2. MNIST dataset

The MNIST dataset contains handwritten digits of 60 000 training set examples and 10 000 test set examples [6]. Each picture in the dataset was flattened to a 784 dimensionality, and the pixel values were scaled to:

$$X = X/255 \quad (2)$$

Essentially scaling them to [0,1]. We split the data in the same ratio as the runge, and set a fixed random state (42) to ensure reproducibility.

This reprocessing part ensures that the different tasks in this project uses the same settings across all the parts.

We will use our neural network to study and analyze both the regression and classification problems, comparing the results against python libraries like PyTorch. Different activation functions will be evaluated, as well as testing a broad specter of hyperparameters.

B. Feed Forward Neural Network

FFNNs has a architectural design where input data flow from input layer, through hidden layers to output layer. In input layer the number of neurons (nodes) corresponds to the number of features in input dataset. Number of hidden layers and neurons within them are customizable and considered hyperparameters in the model architecture. The output layer contains a number of neurons equal to the number of target variables, or parameters to be predicted. Each connecting line between neurons in hidden layers represents a weight, and in each node biases and weights are updated by use of an activation function. Type of activation function in hidden and output layer determines of the neural network is used for regression and classification. Learning is achieved by adjusting weights and biases to minimize a cost function using gradient descent optimization methods like ADAM, RMSProp and Stochastic Gradient Descent. This section will describe steps and functions in a FFNN algorithm.

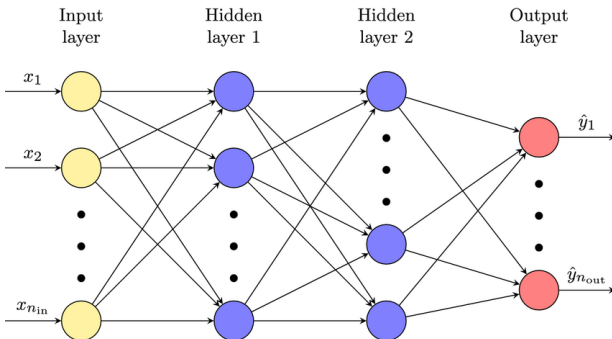


Figure 1: Schematic structure of FFNN [2].

1. Activation functions

Activation functions are used in FFNN to introduce non-linearity, without activation functions, FFNN would actually only be a linear transformations. Hence, activation functions are a vital part of FFNN to learn complex patterns. These functions map and control the output range from input values at each neuron. Activation functions are chosen by their capabilities for regression or classification tasks, and it is normal to have the same activation function in all hidden layers and another activation function in output layer.

The sigmoid function map input values to the range (0,1) and can be considered as probabilities.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

In figure 2 the x-axis represents input values in range (-5, 5) and y-axis represents values from sigmoid function. The blue line is the sigmoid function. Red points exemplify input values to a neuron that is mapped onto the sigmoid function. The steepest part of the curve is around 0, smaller values of x give large change in activation function. For very small or very large input values this function can lead to vanishing gradient problem. With output from function very close 0 or 1 the update of weights become negligible and learning slows down, stops or never converge. Sigmoid is used in binary classification, or when multiple targets can be true in a multi-label classification.

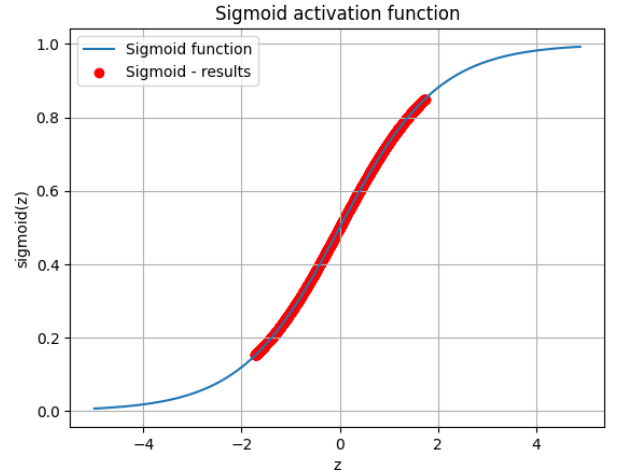


Figure 2: Sigmoid function and mapped values.

The softmax activation function is given by:

$$s_i = \frac{e^{z_i}}{\sum_{i'=1}^n e^{z_{i'}}}$$

where s_i is the softmax output (probability for class i), z_i is logit (raw score before activation), and n is the number of classes. The function is used to take a vector of any

size to probabilities. The exponential ensures positive numbers only and normalization ensures that the vector sums up to 1 [5]. Softmax is usually used in multi-class classification when only one target is true.

A simpler activation in mathematical terms is the ReLU function. Any negative input value is mapped to 0, while any positive input value is identical for output value. Hence, the range is $(0, \infty)$. This activation function is quite common in neural networks and avoid vanishing gradient problem for positive values. For negative values neurons can be stuck at 0 and neurons are inactive and stops learning (dying ReLU).

$$f(x) = \max(0, x)$$

Leaky ReLU can avoid dying ReLU with a bit more complex function where a small term α is added to negative input values. Value of α must be chosen with care, with too small value will behave like ReLU and too large can distort training.

$$f(x) = \begin{cases} x & \text{if } x > 0, \\ \alpha x & \text{if } x \leq 0 \end{cases} \quad (3)$$

For the output layer in regression tasks a linear activation function is typically used. The output is a direct linear combination of the inputs, meaning it is the weighted sum of the input feature and biases. This ensures that the output can have any real number.

2. Regularization terms

Regularization terms are used in machine learning to punish overly large weights, which can lead to overfitting. $L1$ and $L2$ are two commonly used regularization terms that add a penalty to the cost functions [3].

$$J(w) = J_{\text{data}}(w) + \lambda \|w_1\|_p \quad (4)$$

[3]

Where J_{data} is the data loss and λ are hyperparameter that controls the strength of the regularization.

$L1$ regularization, also known as the Lasso regularization, adds an absolute value for the weights, resulting in small and insignificant values turns to zero. $L2$ regularization, or Ridge regularization, adds a squared value to the weights, hindering scaling problems where a single weight becomes very large. [3]

In the gradient computation, **L1** and **L2** modify the weight update as follow:

$$\frac{\partial J}{\partial w_i} = \frac{\partial J_{\text{data}}}{\partial w_i} + \lambda_1 \text{sign}(w_i) + \lambda_2 w_i \quad (5)$$

[3]

Where the regularization terms pushes the weights towards zero, which reduces the model complexity and improves generalization on unseen data.

3. Cost functions

Cost functions measure how well a predicted value from the forward pass fits the target values. The regularization terms, $L1$ and $L2$, are implemented into the cost functions to constrain the size of the weights [4]. Common cost functions are Mean Squared Error for regression and cross-entropy for classification. The derivative of the cost function is used in back propagation. The Mean Square Error is given by

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum_{i=0}^{n-1} (y_i - \tilde{y}_i)^2, \quad (6)$$

where y are the data and $\tilde{y}$ are the predictions. $L1$ and $L2$ penalty term is added to the cost function

$$\mathcal{L}_{\text{MSE}}^{\text{reg}} = \mathcal{L}_{\text{MSE}} + \lambda_1 \sum_j |w_j| + \lambda_2 \sum_j w_j^2 \quad (7)$$

The MSE describes the mean of the squared differences between the model and the data. The smaller the MSE is, the closer the predicted values are to the real ones.

The categorical cross-entropy is given by:

$$\mathcal{L}(y, s) = - \sum_{i=1}^c y_i \cdot \log(s_i)$$

where y_i is the true label for class. s_i is the predicted probability for class after a sigmoid activation function has been applied. c is the number of classes. In one-hot encoding, the true label is represented as a vector of length c with a value of 1 for the correct class and 0 for all others. Since the sum of a one-hot vector is 1, and its elements can be interpreted as probabilities, the derivative of the loss function with respect to the logits z_j can be expressed as [5].

$$\frac{\partial \mathcal{L}}{\partial z_j} = s_j \cdot \sum_{i=1}^c y_i - y_j = s_j - y_j$$

where z_j is the logit for class j and s_j is the corresponding softmax probability.

4. Forward pass

The forward pass receives input data where each feature corresponds to one neuron in the input layer. The input is propagated forward through neurons in all hidden layers and finally to the output layer, as shown in figure 3.

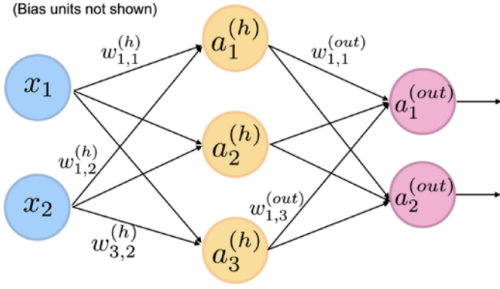


Figure 3: Forward pass for a NN

[7]

In each neuron the weighted sum of inputs plus a bias term is computed, and chosen activation function is applied to introduce non-linearity. This non-linearity enables the network to learn complex patterns. Activations, weights and biases in each layer and neuron are stored for use in backpropagation. The output layer produces predicted values for one or more target variables, which are then passed to the loss function to measure prediction error.

5. Backpropagation

During backpropagation, the network uses the activations, weights and biases stored from forward pass. Starting from the output layer and running backwards, the derivative (gradient) of the chosen cost function is calculated using the chain-rule. The chain rule describes if a quantity L is dependent on another quantity z , which in turn is dependent on w [7]. We can compute the total derivative as:

$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial a} \frac{da}{dz} \frac{\partial z}{\partial w_j} \quad (8)$$

[7]

where

- w_j is the weight
- z is weighted input to a neuron
- a is the neurons activation
- L is the loss function comparing the prediction to the model

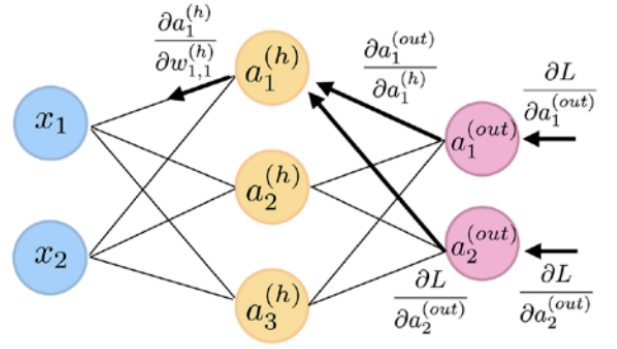


Figure 4: Backpropagation illustration

[7]

$$\frac{\partial L}{\partial w_{1,1}^{(h)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial a_1^{(h)}} \cdot \frac{\partial a_1^{(h)}}{\partial w_{1,1}^{(h)}} + \frac{\partial L}{\partial a_2^{(out)}} \cdot \frac{\partial a_2^{(out)}}{\partial a_1^{(h)}} \cdot \frac{\partial a_1^{(h)}}{\partial w_{1,1}^{(h)}} \quad (9)$$

[7]

These gradients represent how much each weight and bias contribute to the error, and can then determine which values the parameter should be adjusted for to minimize the loss.

The gradient are then used in the weight update step, where each parameter is modified slightly in the opposite direction of the gradient, using learning rate as scaling.

6. Gradient descent methods

Gradient descent methods are used to update each weight and bias using the gradients to minimize the loss function. To accelerate convergence, hyperparameters such as the learning rate and regularization terms are used. Learning rate determines to step size and speeds up convergence, while penalty terms (λ) penalize large weights by pulling these toward zero. Mini-batch can be used to control how noisy the gradient is. Common gradient descent methods used are:

- Stochastic Gradient Descent (SGD): Updates parameters using one training example at a time, introducing noise that can help escape local minima.
- RMSProp: Adapts the learning rate for each parameter by dividing by a running average of recent gradients, improving convergence stability.
- ADAM (Adaptive Moment Estimation): Combines momentum and RMSProp by maintaining running averages of both gradients and their squared values, making it one of the most widely used optimizers.

The choice of gradient descent methods is essential in the neural network training procedure. In this project, ADAM was primarily used for the building of the neural

network, as it provides running averages of both gradients and their squared values. While the RMSProp and SGD were used during the classification to compare optimizers better suited for noisy gradients.

7. Repeat

The repeat step refers to the repetitive nature of the neural network. After the model has gone through forward pass $\rightarrow$ backpropagation $\rightarrow$ parameter update with gradient decent, the model repeats itself multiple times. Doing this, while updating the weights each time, the loss will eventually converge at a satisfactory level or stop at the given maximum iterations. The number of iterations acts as a hyperparameter for how long the network can train. Too few iterations can lead to underfitting, while too many iterations can in turn lead to overfitting. Shuffling of input data are performed for each iteration, sorting data in a random order each iteration. Then batches of size B are feed through forward pass and back propagation.

8. PyTorch

PyTorch is a scalable and multiplatform programming interface used to implement and run machine learning algorithms [7]. We used PyTorch as reference implementation to compare with our own neural network. This was implemented as a framework for both the Runge regression model and the MNIST classification.

The datasets were first converted from NumPy-arrays to PyTorch-Tensors in, and then further packed into Dataloader and TensorDataset for MNIST to enable minibatch-training and shuffling of training data. The OLS design matrix and splits from our neural network was imported and converted to torch.Tensor.

We trained two hidden layers with sigmoid in the hidden layers and a linear output. One layer with two hidden layers and [50, 100] nodes, while the other had one hidden layer and [50] nodes, using 256 mini-batches and 100 epoches, with optimizers SGD, ADAM and RMSProp with no regularization terms and $\eta \in [10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}]$. Finally, we plotted the two respective models into heatmaps.

9. Pseudocode for own implementation of FFNN

Algorithm 1 Feed Forward Neural Network

Procedure Train Feed Forward Neural Network

Input Training data $X_{\text{train}}, Y_{\text{train}}$. Validation data $X_{\text{test}}, Y_{\text{test}}$

Hyperparameters Learning rate η , regularization λ , number of epochs E , batch size B , optimizer (SGD/RMSProp/ADAM)

Initialize Weights W and biases b randomly by normal distribution.

for each epoch **do**

Shuffle randomly over whole training dataset

for each batch **do**

Split shuffled data into mini-batches by size B

Feed forward through all layers to obtain predictions $\hat{y}$

Compute loss (including regularization term) based on task (MSE for regression, cross-entropy for classification).

Backpropagation to compute gradients ∇_W, ∇_b

Update weights and biases using chosen optimizer (gradient descent)

end for

Compute training loss and validation metric (MSE or accuracy) on X_{train} and X_{val}

Compute predictions on X_{test}

end for

End Procedure

C. Use of AI tools

In this project, we have used AI tools Microsoft Copilot in the following ways in the production of the code and the report:

- Questions about python syntax.
- General questions about Overleaf codes
- Enhancing sentences in report.
- Question about cost functions available at: Github Repository

III. RESULTS & DISCUSSION

A. Regression Results and Analysis

To analyze the Runge function, we construct a design matrix with polynomial degrees up to 15. A polynomial of degree 15 achieved the lowest MSE and was also used in Project 1 [9]. For the FFNN, we typically use one input feature corresponding to the x -values from the Runge function, but here we do so to ensure that the analysis is based on the same dataset.

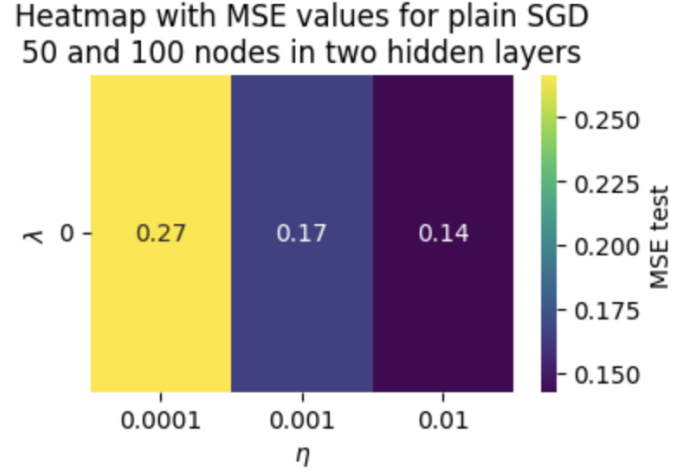
The dataset was divided into 80% training and 20% test data, followed by standard scaling. For consistency with Project 1, where $p = 15$ was also used, we set the polynomial degree to 15 as our baseline [9].

1. Assess different optimizers in FFNN

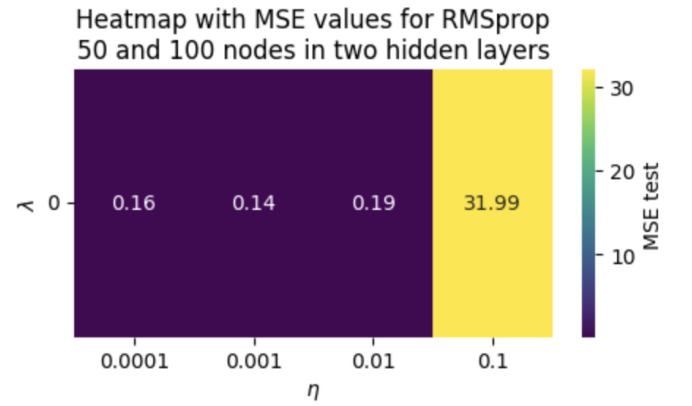
We used FFNN to analyze the Runge function in several steps. We trained our model with $L1/L2 = 0$ to isolate the effect of the optimizers and learning rate. We use sigmoid for the hidden layers and a batch size of 256 with a 100 iterations. We tested two different neural network architectures, one with two hidden layers and (50, 100) nodes. Where we trained the architecture with three optimizers (SGD, ADAM, RMSProp) over the learning rates $\eta \in [10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}]$.

Since these models have a $\lambda = 0$, the performance is purely based on the learning rate. Low values of η leads to underfitting and slow learning rate, middle values leads to stable MSE values, while large steps results in the MSE values overshooting as we see in figure 5a 5b and 5c.

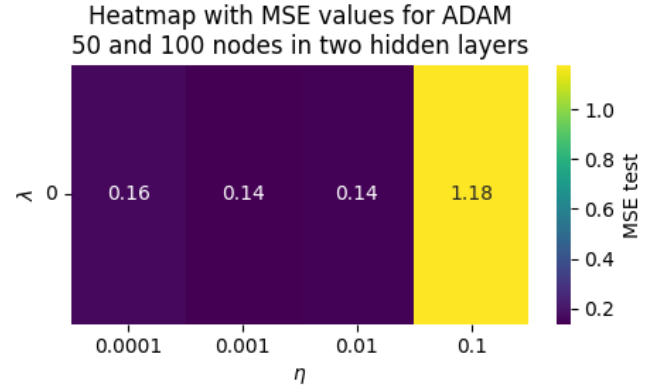
We saw that the results for the model with two layers and (50, 100) nodes yielded the best performance with an MSE of 0.14 for SGD at $\eta \in 0.01$. RMSProp achieved a similar result with 0.14 MSE at $\eta \in 0.001$, while ADAM reached an MSE of 0.14 for both $\eta \in 0.001$ and $\eta \in 0.01$. High learning rates $\eta = 10^{-1}$ caused unstable MSE values and divergence, where RMSProp and ADAM yielded higher values of 31.99 and 1.18. This is expected, as Raschka notes that the MSE will diverge when the learning rate becomes too large [7].



(a) Heatmap showing two hidden layers for SGD.



(b) Heatmap showing two hidden layers with [50, 100] nodes for RMSProp.

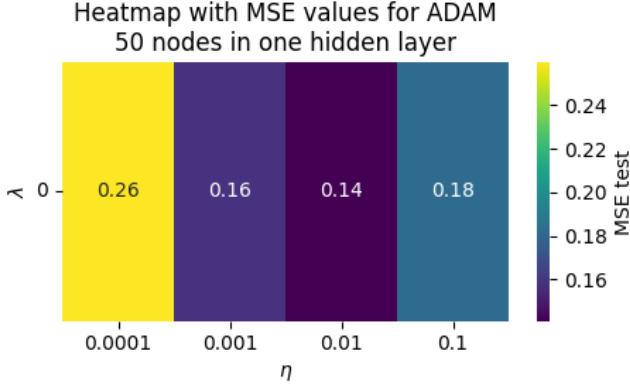


(c) Heatmap showing two hidden layers with [50, 100] nodes for ADAM

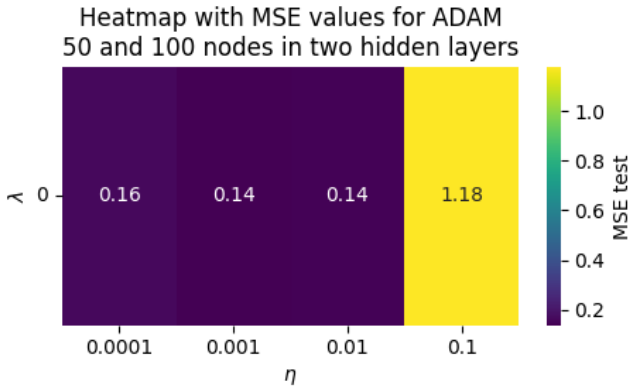
Figure 5: The heatmaps are based on our own FFNN coded

Although it is expected that the MSE becomes unstable at higher learning rates, we observe that RMSProp diverges much higher than ADAM. This can be due to

ADAM's use of both momentum and adaptive learning rates, which help stabilize the updates even under larger gradient magnitudes. RMSProp, on the other hand, only adapts the learning rate based on recent gradient variance and lacks the momentum term that helps dampen oscillations. As a result, ADAM remains more robust to large η -values, while RMSProp becomes more sensitive and prone to gradient explosion.



(a) Heatmap showing one hidden layer with 50 nodes



(b) Heatmap showing two hidden layers with [50, 100] nodes

Figure 6: Both heatmap are based on our own FFNN coded, and uses ADAM as optimizer.

ADAM generated the best results in terms of MSE for the heatmaps. Figure 6a, represent a FFNN with one hidden layer and 50 nodes. We see that the model learns slowly and slightly underfits at small values of η ($MSE = 0.26$). The error drops after $\eta = 0.001$, and reaches a minimum value of 0.14 when $\eta = 0.01$. Increasing to $\eta = 10^{-1}$ hurts again ($MSE = 0.18$), which complies with overshooting/instability.

Figure 6b shows two hidden layers with 50 and 100 nodes. This model is more stable across the smaller learning rates, showing $MSE = 0.16$ for $\eta = 10^{-4}$ and $MSE = 0.14$ for both $\eta = 10^{-3}, 10^{-2}$, indicating a plateau, and that the deeper model is less sensitive to learning rates when they are in a reasonable range.

When η increases to 10^{-1} , the model becomes unstable and MSE spikes to (1.18). Illustrating that the model diverge when steps are too large.

Both heatmaps achieve the best MSE value at $\eta = 10^{-1}$. The deeper model is more robust, but fails at $\eta = 10^{-1}$.

Figure 7 illustrates the Runge function, and our own neural network predicted values for OLS, along with 3 different optimizers for plain SGD, RMSProp and ADAM. The neural network uses two hidden layers with [50, 100] nodes. The predictions is based on the parameters that produced the best MSE values to the corresponding heatmaps ??, 6a.

As we see in figure 7, all functions performed similarly at $x = 0$, where the curves follow the Runge function closely, because the gradients here are easier to approximate. The edges on the functions, on the other hand, ($x > 0.75$), display a more scattered range of curves. This is because the Runge function near the boundaries is harder to capture due to higher sensitivity near the boundaries. All the FFNN models show oscillation in this region, although ADAM shows slightly less oscillation, followed by RMSProp and SGD. OLS is able to capture the general shape of the Runge function, but struggles to capture the more demanding section at the edges. Showing larger bias and deviation from the curve, illustrating its limited flexibility.

Compared to the OLS, ADAM and RMSProp proves to be more capable of capturing the interval, which is due to their ability to adjust the learning rate based on gradient history. Both optimizers smooth out the oscillations near the edges compared to SGD.

Plain SGD, which use a fixed learning rate and no momentum, shows a slightly more variation in the edges, which is explained by the lack of flexibility in learning rate. To close in on ADAM and RMSProp stability, SGD probably need more epochs and fine tuning of learning rate.

RMSProp and ADAM close similarity along the curve suggest that both optimizers FFNN architectures are robust and deliver adequate model capacity for this regression task. The near identical curve for ADAM and RMSProp demonstrates a strong generalization. Showing that the network does not memorize the data and capture the Runge function.

To summarize, figure 7 illustrate the importance of a deeper network to handle non-lineared problems like the Runge function, particular along the edges, where the gradient variance changes.

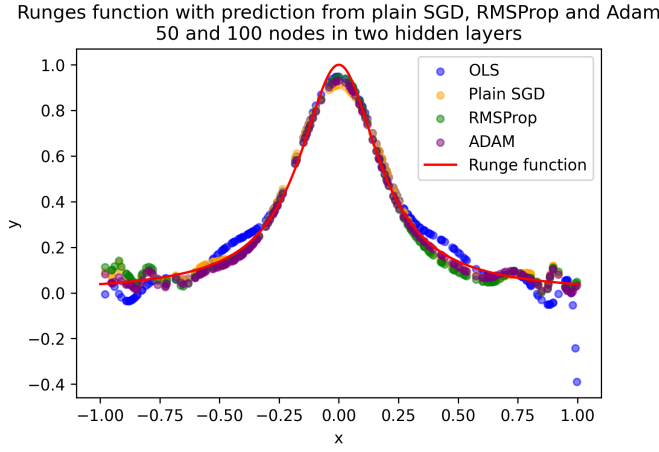


Figure 7: Runge function with predictions for OLS, plain SGD, RMSProp and Adam optimizers. Predictions based on best MSE value from corresponding heatmap. FFNN used for calculations.

2. Validation of FFNN

We benchmarked our own code for FFNN versus PyTorch library by using the same input data, architecture and hyperparameters, as seen in figure 8.

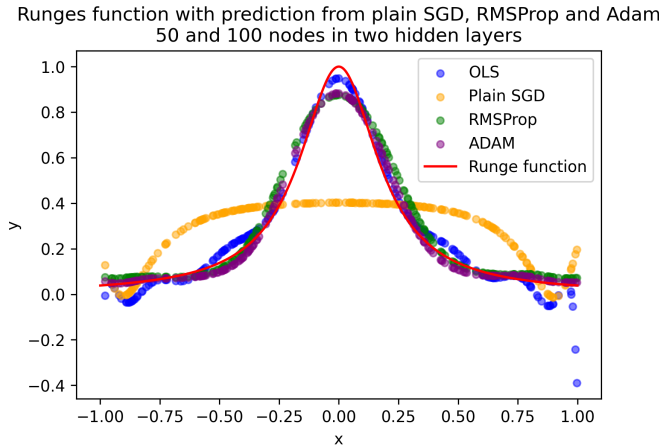


Figure 8: Runge function with predictions for OLS, plain SGD, RMSProp and Adam optimizers. Predictions based on best MSE value from corresponding heatmap. PyTorch used for calculations.

In contrast to our model, plain SGD underfits strongly, the peak is flattened and the edges are pulled toward zero. The curve also sits too high for $(x \in [0.3, 0.75])$, indicating strong bias and insufficient capacity/optimization in practice. This behavior is consistent with slower, noisier convergence for plain SGD and a tendency to settle in flatter regions of the loss.

RMSProp captures the overall shape very well. Al-

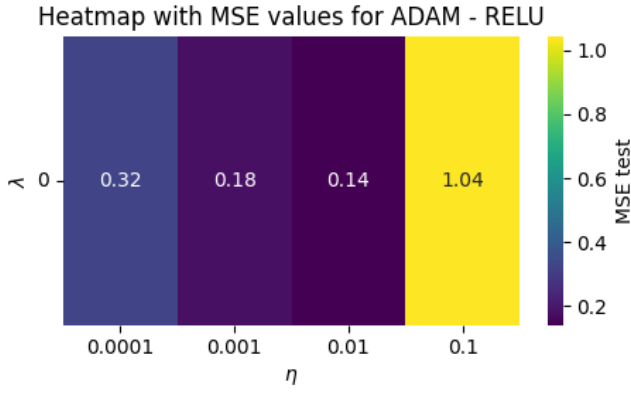
though it has a marginally lower peak than our model, the edges flatten out more and are less noisy, while our model showed more scatter near the boundaries. This suggests a slightly stronger smoothing effect from RMSProp (lower variance at the edges), at the small cost of a modest bias at the peak. Overall, the bias variance trade-off is clearly preferable to plain SGD and competitive with our implementation.

Adam yields a near-identical curve to RMSProp and captures the overall shape well. As with RMSProp, the peak is slightly lower than in our model, but the edges are flatter and exhibit less noise, reducing the scatter observed at the boundaries. In addition, Adam's predictions appear the most stable across the domain, which aligns with its faster and more robust convergence on this task.

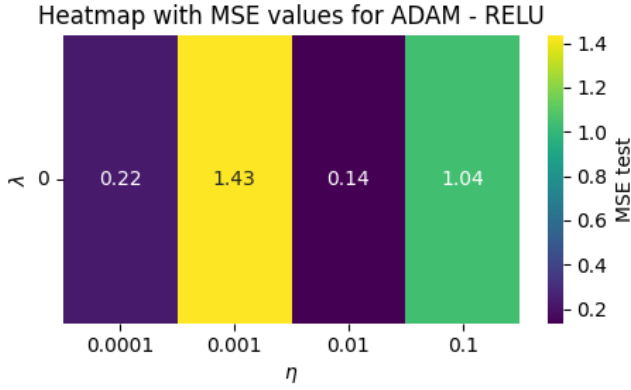
To summarize, PyTorch RMSProp and ADAM performed better along the edges and poorer on the peak than our model, delivering smoother and less scattered points. This demonstrates PyTorch better capability to utilize gradient history compared to our FFNN. While the SGD performed poorly with a strong underfitting, showing that its critical with a flexible learning rate with PyTorch.

3. Testing different activation functions and depths of the neural network

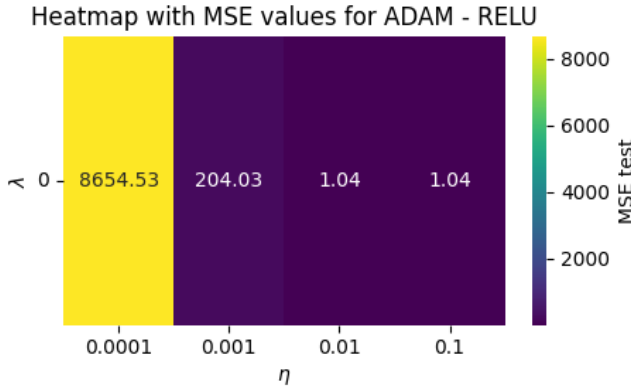
Figure 9 represents heatmaps of MSE values using the RELU activation function with networks of varying depth. With one hidden layer the stability of training is fairly good, and few layers can yield good training for simple regression tasks. With two hidden layers instability of training are prominent between learning rates, more hidden layers leads to sensitivity for learning rates. Five hidden layers gives extreme variations, and exploding MSE value for small learning rate. The RELU activation function can lead to exploding gradients and overshooting minima, especially for small learning rates. Even small learning rates can accumulate gradient errors and cause divergence.



(a) Heatmap of ReLU activation function and 8 nodes in one layer



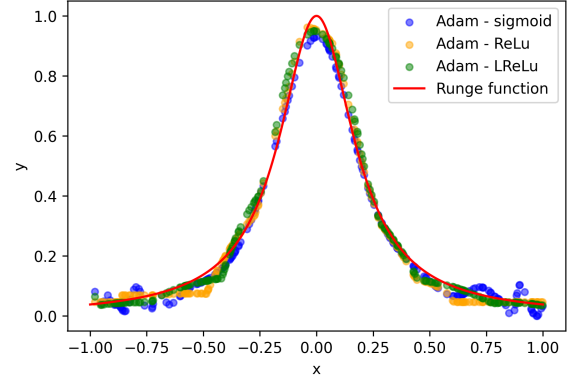
(b) Heatmap of ReLU activation function and 50, 100 nodes in two hidden layers



(c) Heatmap of ReLU activation function and 50, 100, 50, 100, 50 nodes in five hidden layers

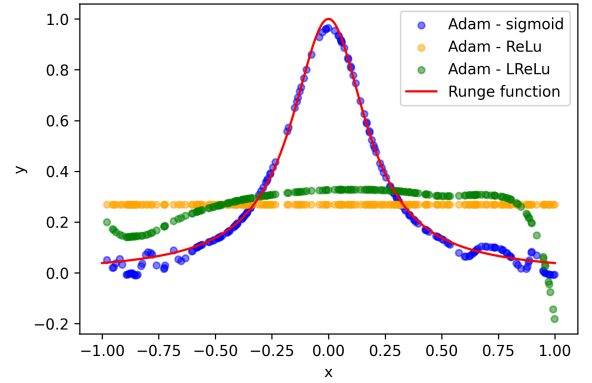
Figure 9: Heatmaps with ReLU activation function and different nodes and number of hidden layers

Runge function with Adam predictions with different activation functions 50 and 100 nodes in two hidden layers



(a) Predicted values based on different activation functions. 50 and 100 nodes in two hidden layers

Runge function with Adam predictions with different activation functions 50, 100, 50, 100, 50 nodes in FIVE hidden layers



(b) Predicted values based on different activation functions. 50, 100, 50, 100 and 50 nodes in five hidden layers

Figure 10: Predicted values based on different activation functions and number of hidden layers.

Figure 10 illustrates predictions obtained using FFNN trained with ADAM optimizer and sigmoid, LReLU and ReLU activation functions. For each activation function, we used the best MSE and corresponding learning rate. With two hidden layers, ReLU and LReLU activation function have distinct critical points where the predictions change direction abruptly. Sigmoid activation function produces fairly smooth prediction close to the target function. With five hidden layers ReLU and LReLU fails to converge and predictions are nearly constant. ReLU and LReLU do not constrain output, which can lead to exploding gradients and weights and poor generalization. Sigmoid do constrain output between 0 and 1, and avoid growing gradients and stabilize training even with unnecessary many hidden layers.

4. Testing different norms

To improve generalization and avoid overfitting we introduce L1 (Lasso) and L2 (Ridge) regularization term to the FFNN training process. Results are analyzed to assess how these contribute to model performance and to compare against results with Ridge and Lasso in project 1.

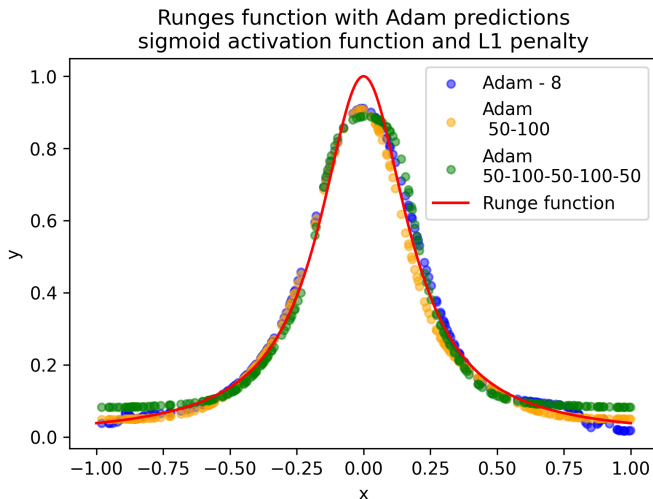


Figure 11: FFNN with sigmoid activation function. One, two and five hidden layers with number of nodes in figure legend.

Figure 11 illustrates predictions using Adam optimizer, sigmoid activation and L1 penalty across three network configuration with increasing depth. The corresponding plot for L2 penalty shows similar behavior, with only slightly poorer performance near the edges of domain. Hence, both L1 and L2 improves generalization. L1 penalty drives weights toward zero which simplifies network, leading to smaller gradient updates and smoother predictions. This is most prominent at the edges where function values changes rapidly. L2 has less aggressive reduction of gradients where function values changes rapidly, and explain the slightly worse performance.

We compared these results to Ridge (L2) and Lasso (L1) regression in project 1, again ensuring that we used the dataset. By comparing heatmaps from Lasso regression and FFNN with L1. The MSE values are in general lower and more consistent across learning rates and lambda values with Lasso regression than FFNN, but the lowest MSE difference is only 0.02. By visually inspecting figures of predicted values it is clear that FFNN smooth out the edges better.

Both OLS and Ridge (L2) regression were in project 1 done analytically. In contrast, Lasso regression require gradient descent due to non-differentiable absolute values. According to project 1 OLS has an MSE value of 0.14, while Ridge performed significantly better with an MSE value of 0.033 using penalty $\lambda = 0.01$. Lasso out-

performed both, and notably achieved better results than the feedforward neural network (FFNN), which had an MSE of 0.18. OLS also slightly outperformed the FFNN. Ridge benefits from its penalty term which effectively smooth oscillations at edges better than OLS. This shows that FFNN is unnecessary complex and prone to overfitting for regression tasks of linear functions. For simpler prediction tasks, analytical regression methods such as OLS and Ridge are more appropriate and efficient.

B. Classification results (MNIST)

The MNIST database of handwritten digits has a training set of 60 000 examples, and a test set of 10 000 examples [6]. We applied min-max scaling to convert pixel ranges from $[0, 255]$ to $[0, 1]$ which helps to stabilize training and faster convergence. The accuracy metric for multi-classification is

$$\text{Accuracy} = \frac{\sum_{i=1}^n I(t_i = y_i)}{n},$$

where $I(t_i = y_i)$ is an indicator function that equals 1 if the predicted label y_i matches the true label t_i , and 0 otherwise.

The learning rates considered were $\eta \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}\}$, and the regularization parameters were $\lambda \in \{10^{-2}, 10^{-2.67}, 10^{-3.33}, 10^{-4}\}$. Furthermore, the activation functions ReLU, Leaky ReLU, and sigmoid were evaluated.

1. FFNN

The FFNN classified the same accuracy score of 90 across all learning rates and λ values as shown in 12. The corresponding depth of network and hyperparameters with PyTorch implementation can be seen in 13b with variation of accuracy across learning rate and λ values. The FFNN has performed close to PyTorch implementation for regression analysis, so there might be a problem with implementation of cross-entropy loss function or its integration to training loop.

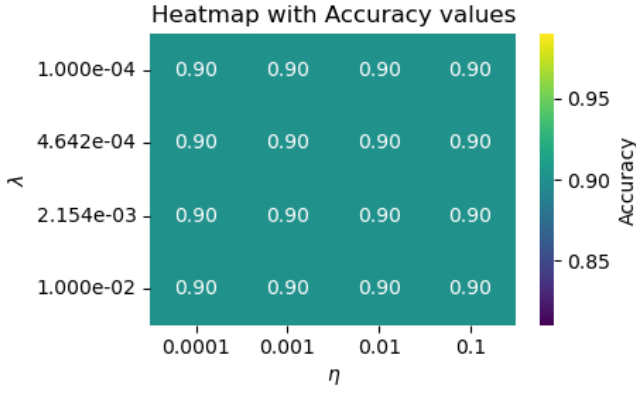
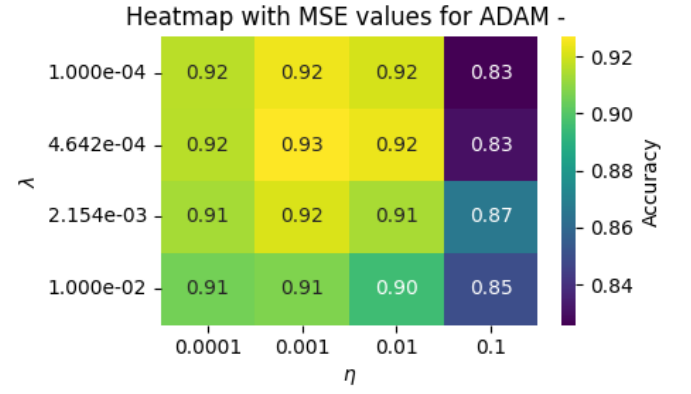


Figure 12: FFNN with 50 and 100 nodes in two hidden layers. RELU activation function.



(a) PyTorch with 8 nodes in one hidden layer. L1 penalty - RELU activation.

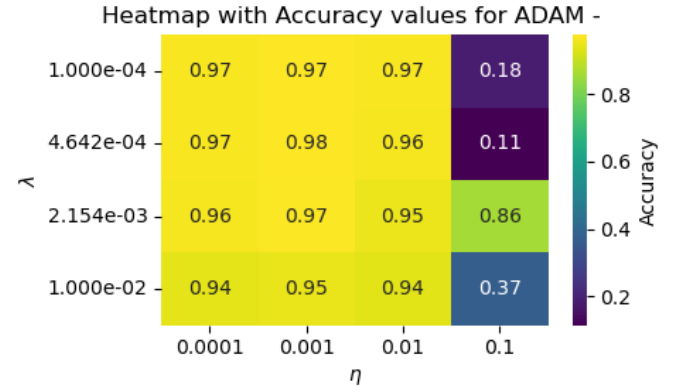
2. PyTorch benchmark

We classified the MNIST dataset using PyTorch as a benchmark for FFNN. Despite a shallow network of 8 nodes in one hidden layer, the model achieved an accuracy of 92% when using the RELU activation function.

We generated heatmaps showing accuracy as a function of learning rate, penalty term and activation function. Figure 13a show L1 regularization with RELU activation function, the corresponding heatmap for L2 regularization is identical in terms of MSE values. Heatmaps with LRELU L1 and L2 penalty are also identical, and close to identical for RELU.

Softmax on the other hand shows more instability across learning rates and penalty terms. Heatmaps for L1 and L2 also here have exact same numbers. Softmax is typically used for output layers in multi-classification, using this in hidden layers can lead to vanishing gradients and poor convergence.

The regularization is expected to be more important for the Runge function scenario, since the MNIST dataset is expected to have less significant noise and oscillations in the dataset. The accuracy is consistent across learning rates and regularization, which further indicate low noise level in data. Consequently, for datasets with little noise, choice of activation function and regularization term is less critical to overall performance.



(b) PyTorch with 50 and 100 nodes in two hidden layers. L1 penalty - RELU activation.

Figure 13: Heatmaps from PyTorch classification of MNIST dataset with different network depth.

3. Classification with logistic regression

Softmax regression (multinomial logistic regression) is generalized logistic regression for classifying multiple classes. This method work well when data are linearly separable [8]. For both FFNN and softmax regression the input data are flattened into a vector. 28x28 pixels are flattened to a 784-dimensional vector. Flattening leads to lost spatial structure, but not pixel value information as features. Softmax regression uses only features and would reduce the dimensionality and convergence time for classifying MNIST.

Softmax regression gave accuracy score of 92% with sklearn in significantly less processing time than for FFNN and PyTorch with same dataset.

C. Summary of analysis

1. Regression of Runge function

An custom FFNN were implemented to do both regression and multi-class classification. FFNN were benchmarked versus PyTorch and show similar results, however documenting some differences in implementation.

The regression analysis was performed on the same Runge function and dataset as in project 1 to perform a comparative study of the results of project 1[9]. This means that we used a design matrix with polynomial degree 15, while for FFNN we would normally use one feature on input data and one target. Data are scaled with standard scaler, that is normalization where each feature are centered by subtracting the mean and scaled by standard deviation.

The lowest MSE values are achieved with analytical Ridge regression (L2 penalty), emphasizing that simpler analytical approaches than FFNN should be used for linear tasks.

By gradually adding hyperparameters in our study we revealed how these influence the predictions. First we compared optimizers plain SGD, RMSProp and ADAM with the same range of learning rates (no penalty). All with sigmoid activation function and 50, and 50 and 100 nodes in a network with one and two hidden layers respectively. These results that learning rate optimizers RMSProp and ADAM has less instability across learning rates. ADAM uses both learning rate and momentum which dampens the oscillations at the edges of Runge function.

By introducing different activation functions we found that sigmoid controls gradients by constraining output between 0 and 1, while RELU and LRELU get exploding gradients and overfitting with deep networks. Increasing cross learning rate instability were shown with RELU activation function and increasing depth of network.

By introducing norms to the FFNN we showed that this significantly improve generalization where data are noisy, visualized as smoother edges of Runge function. L1 is slightly better than L2 since it has more aggressive reduction of gradients.

2. Classification of MNIST dataset

We used the MNIST database to do multi-classification of handwritten numbers.

First we used FFNN to classify MNIST dataset and

compare to PyTorch implementation. There are differences between these two results that might show a problem with implementation of cross-entropy loss function or it's integration to training loop for FFNN. With one layer and 8 nodes best accuracy score were 93, while with 50 and 100 nodes in two layers we get accuracy score of 98. Learning rate 0.1 is clearly to low and cause instability in training and not converging training.

Difference between LRELU and RELU are minor. Heatmaps with L1 and L2 regularization are identical, telling us that penalization terms have little impact on model performance. This is likely the low noise level in MNIST dataset.

IV. CONCLUSION

The custom FFNN implementation performed comparably to PyTorch for regression tasks, confirming the correctness of core components such as gradient descent and activation functions. However, MNIST classification revealed inconsistencies, likely due to issues with the cross-entropy loss function or its integration into the training loop. Hyperparameter tuning, learning rate, activation functions and penalty terms had a significant impact on performance for regression tasks. While activation function choice (ReLU vs. LReLU) and regularization (L1 vs. L2) showed minimal influence on MNIST due to its low noise level. Softmax is commonly used in output layer for multi-classification, but proved to be not applicable for hidden layers. For regression on the Runge function, analytical Ridge regression achieved the lowest MSE, emphasizing that simpler analytical methods outperform FFNN for linear tasks.

Further work needs to be done to ensure that multi-classification of FFNN works properly and give expected results. Since PyTorch and FFNN give slightly different results for regression, further studies on FFNN implementation can be done to increase knowledge and improve the neural network training. More complex functions and dataset can be used to understand performance of FFNN better.

V. CODE AVAILABILITY

The code for this project is available at: GitHub Repository Link - project 2

-
- [1] W. contributors. Runge's phenomenon. https://en.wikipedia.org/wiki/Runge%27s_phenomenon. Accessed: 2025-10-29.
 - [2] D. Coscia, L. Meneghetti, N. Demo, G. Stabile, and G. Rozza. A continuous convolutional trainable filter for

modelling unstructured data. *Computational Mechanics*, 72:1–13, 03 2023.

- [3] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, <https://www.deeplearningbook.org/>, 2016.
- [4] M. Hjorth-Jensen. Fys-stk4155 - lecture notes week 42,

- 2025.
- [5] T. Kurbiel. Derivative of the softmax function and the categorical cross-entropy loss, 2021. Accessed: 2025-11-04.
 - [6] Y. LeCun, C. Cortes, and C. J. Burges. The mnist database of handwritten digits. <https://www.kaggle.com/datasets/hojjatk/mnist-dataset/data>.
 - [7] S. Raschka, Y. Liu, and V. Mirjalili. *Machine Learning with PyTorch and Scikit-Learn*. Packt Publishing, <https://sebastianraschka.com/blog/2022/ml-pytorch-book.html>, 2022.
 - [8] A. Saha. Comparison between logistic regression and neural networks in classifying digits.
 - [9] J. Tjøtta. Data analysis and machine learning projects – assignment 1. https://github.com/johtj/data_analysis_ml_projects/tree/main/assignment1, 2023. Accessed: November 10, 2025.